

Prob 1. (20 pts total) Write an assembly function to implement the following C function:

```

1 @ int mp_selector(int a, int x, int y) {
2 @     if (a >= 0) {
3 @         return x;
4 @     } else {
5 @         return y;
6 @     }
7 @ }
```

Clearly specify all the registers you use in a similar way as you see in various examples.

Solution:

```

1 @ int mp_selector(int a, int x, int y) {
2 @     if (a >= 0) {
3 @         return x;
4 @     } else {
5 @         return y;
6 @     }
7 @ }
8 @ int mp_selector(int a, int x, int y)
9 @     Input:
10 @         * a -> r0, x -> r1, y -> r2
11 @     Output:
12 @         * r0
13 mp_selector:
14     cmp     r0, #0
15     blt     mp_selector_else
16 mp_selector_then:
17     mov     r0, r1
18     bx      lr
19 mp_selector_else:
20     mov     r0, r2
21     bx      lr
```

:-

Prob 2. (30 pts total) Determine the operation results for (a) unsigned in decimal format, (b) signed in decimal format, (c) final result in binary, and (d) the N, Z, C, and V flags of the following calculation for a **5-bit** system using the C and V flag convention of an ARM processor: (1) (-15) - (-16); (2) 16 - 18

Soln:

For a 5-bit system, we use the following constants: (1) $C_N = 32$, (2) $U_{\min} = 0$, (3) $U_{\max} = 31$, (4) $S_{\min} = -16$, and (5) $S_{\max} = 15$.

Part (1): (-15) - (-16)

(a, 7pts) Unsigned. $S_{\text{temp}} = (-15 + 32) - (-16 + 32) = 17 - 16 = 1 \in [U_{\min}, U_{\max}] \Rightarrow$

- Decimal result = 1 (4 pts)
- B = 0 $\Rightarrow$ C flag = 1 (2 pts)
- Z flag = 0 (1 pts)

(b, 7 pts) Signed. $S_{\text{temp}} = (-15) - (-16) = 1 > S_{\min} \Rightarrow$

- Decimal result = 1 (4 pts)
- V flag = 0 (2 pts)
- N flag = 0 (1 pts)

(c, 1 pt) Result in binary = 0b00001.

Part (2) 16 - 18

(a, 7pts) Unsigned. $S_{\text{temp}} = 16 - 18 = -2 \notin [U_{\text{MIN}}, U_{\text{MAX}}] \Rightarrow$

- Decimal result = $-2 + 32 = 30$ (4 pts)
- B = 1 $\Rightarrow$ C flag = 0 (2 pts)
- Z flag = 0 (1 pts)

(b, 7 pts) Signed. $S_{\text{temp}} = (16 - 32) - (18 - 32) = -16 - (-14) = -2 \in [S_{\text{MIN}}, S_{\text{MAX}}] \Rightarrow$

- Decimal result = -2 (4 pts)
- V flag = 0 (2 pts)
- N flag = 1 (1 pts)

(c, 1 pt) Result in binary = 0b11110.

∴

Prob 3. (50 pts) Assume we have the following memory map:

Address	Byte 0	Byte 1	Byte 2	Byte 3
0x2000_0000:	0x80	0x81	0x82	0x83
0x2000_0004:	0x84	0x85	0x86	0x87
0x2000_0008:	0x78	0x79	0x7A	0x7B
0x2000_000C:	0x7C	0x7D	0x7E	0x7F

Also assume we have the following ASM functions:

1	task1:		task2:
2	ldr r2, [r0, #4]!		ldr r3, [r0], #8
3	str r2, [r1, #0]		str r3, [r1, #0]
4	ldr r3, [r0, #4]!		ldr r3, [r0, r2, lsl, #2]
5	str r3, [r1, #4]		str r3, [r1, #4]
6	bx lr		bx lr

which are called in C with the following code:

1	void *p1 = (void *)0x20000000;
2	void *p2 = (void *)0x20000020;
3	void *p3 = (void *)0x20000030;
4	task1(p1, p2);
5	task2(p1, p3, -1);

1. (10 pts, 5 each) What are the values of **R2 and R3** just before returning from task1 (assume we have a breakpoint at `bx lr`)?
2. (10 pts, 5 each) What are the values of **R0 and R3** just before returning from task2.
3. (10 pts) Draw the memory map for **8 bytes** starting at **0x2000_0030** as shown in the form given above.
4. (10 pts, 5 each) Write the header file prototypes of the above two assembly functions. Use the **intended** datatype.
5. (10 pts) Translate `task1` into a C function using the prototype you just wrote.

Solution:

Part 1.

- `ldr r2, [r0, #4] ! ==> r2 = 0x8786_8584. (r0 = 0x2000_0004)`
- `ldr r3, [r0, #4] ! ==> r3 = 0x7B7A_7978.`

Part 2.

- `ldr r3, [r0], #8 ==> r0 = 0x2000_0008, r3 = 0x8382_8180`
- `ldr r3, [r0, r2, lsl, #2] ==> r0, r2, lsl, #2 = 0x2000_0004, r3 = 0x8786_8584.`

Part 3.

We need to consider the store operations in `task1`:

Address	Byte 0	Byte 1	Byte 2	Byte 3
0x2000_0030:	0x80	0x81	0x82	0x83
0x2000_0034:	0x84	0x85	0x86	0x87

Part 4.

We have the following prototypes:

- `void task1(int *p1, int *p2);`
- `void task2(int *p1, int *p2, int n);`

Part 5.

```
1 void task1(int *p1, int *p2) {
2     *p2 = *++p1;
3     * (p2+1) = *++p1;
4 }
```

:-